

Práctica de Planificación

Práctica 3



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Antonia Laura Apolzan

Ariadna Mantilla

Guillem Sturlese

2024/25 Q2

Índice

1. Descripción del enunciado.....	3
2. Modelado del dominio.....	4
2.1. Variables.....	4
2.2. Predicados.....	4
2.3. Acciones.....	6
3. Modelado de los problemas a resolver.....	7
3.1. Objetos.....	7
3.2. Estado inicial.....	7
3.3. Estado final.....	8
4. Desarrollo de los modelos.....	9
5. Problemas de prueba.....	10
5.1. Nivel básico.....	10
5.1.1. Primer juego de pruebas.....	10
5.1.2. Segundo juego de pruebas.....	11
5.2. Extensión 1.....	13
5.2.1. Primer juego de pruebas.....	13
5.1.2. Segundo juego de pruebas.....	14
5.3. Extensión 2.....	16
5.3.1. Primer juego de pruebas.....	16
5.3.2. Segundo juego de pruebas.....	17
5.4. Extensión 3.....	19
5.4.1. Primer juego de pruebas.....	19
5.4.2. Segundo juego de pruebas.....	21
5.5. Extensión 4.....	22
5.5.1. Primer juego de pruebas.....	22
5.5.2. Segundo juego de pruebas.....	24
5.6. Extensión 5.....	25
5.6.1. Primer juego de pruebas.....	25
5.6.2. Segundo juego de pruebas.....	27

1. Descripción del enunciado

El objetivo principal de esta tercera práctica es modelar y resolver, mediante técnicas de planificación automática y usando el lenguaje PDDL, el problema de **generar menús semanales óptimos** para un bar o restaurante. El menú semanal debe estar compuesto por cinco menús diarios (de lunes a viernes), y cada menú diario consiste en un primer plato y un segundo plato seleccionados de una lista de platos disponibles.

El sistema debe ser capaz de construir menús que cumplan una serie de requisitos y/o restricciones, y que se van incorporando progresivamente en las diferentes extensiones que presenta el problema:

- **Compatibilidad entre platos:** Algunos platos no pueden ir juntos en el mismo menú diario por razones culinarias, de salud, o de preferencias del establecimiento (en nuestro caso, por preferencias que establecemos nosotros).
- **No repetición de platos:** Un mismo plato no puede aparecer más de una vez en el menú semanal, ni como primero ni como segundo.
- **Clasificación de platos por tipo culinario:** Cada plato pertenece a una categoría (por ejemplo, sopa, carne, pescado, pasta, etc.) y, según el enunciado, se debe evitar que el mismo tipo de plato se repita en días consecutivos tanto para primeros como para segundos, aunque sí se permite que el primero de un día y el segundo del siguiente sean del mismo tipo (y al revés, el segundo de un día y el primero del siguiente).
- **Platos obligatorios en días concretos:** El sistema debe permitir forzar que ciertos platos se sirvan en días específicos (por ejemplo, paella el jueves).
- **Control de calorías:** La suma de las calorías de los platos de cada menú diario debe estar dentro de un rango determinado (según el enunciado, entre 1000 y 1500 cal).
- **Minimización del precio:** El sistema debe ser capaz de seleccionar los menús semanales que, cumpliendo todas las restricciones anteriores, tengan el menor coste económico posible.

En las siguientes secciones del documento se detallan el modelado del dominio y de los problemas en PDDL, el desarrollo progresivo de las extensiones propuestas y los resultados obtenidos en los distintos juegos de prueba.

2. Modelado del dominio

2.1. Variables

Las variables se utilizan para categorizar los objetos del dominio, permitiendo una descripción más estructurada y legible de los parámetros en predicados y acciones.

- **plato:** Este tipo agrupa todos los platos posibles que pueden formar parte del menú semanal. Incluye tanto primeros como segundos platos, sin distinción inicial entre ellos (la distinción se realiza luego mediante predicados como `es_primer` y `es_segundo`).
- **dia:** Esta variable se refiere a los días laborables de la semana en los que se va a generar un menú. En el problema, los días se consideran explícitamente de lunes a viernes.
- **tipo:** Este tipo modela la categoría o clase culinaria a la que pertenece un plato. Ejemplos comunes de tipos podrían ser: sopa, ensalada, carne, pescado, crema, etc. Cada plato está asociado a un único tipo culinario.

2.2. Predicados

Los predicados definen propiedades lógicas o relaciones entre objetos. Estos se utilizan para describir el estado del mundo antes y después de ejecutar acciones.

Clasificación de platos:

Estos predicados nos permiten clasificar cada plato según si se sirve como primero o como segundo. Esta distinción nos sirve para generar menús válidos que sigan la estructura esperada (un primer y un segundo plato por día).

- (`es_primer ?p - plato`): Indica que el plato `?p` es un primer plato.
- (`es_segundo ?p - plato`): Indica que el plato `?p` es un segundo plato.

Compatibilidad:

La función de este predicado es asegurar que las asignaciones diarias respeten las restricciones del restaurante.

- (`incompatible ?p1 - plato ?p2 - plato`): Define una relación de incompatibilidad entre dos platos que no pueden estar juntos en el mismo día del menú.

Asignaciones:

- (asignado ?d - día ?p1 - plato ?p2 - plato): Indica que los platos ?p1 y ?p2 han sido asignados al día ?d.
- (menu_dia ?d - dia): Indica que ?d forma parte de los días a planificar.
- (menu-asignado ?d - dia): Indica que ya se ha asignado un menú al día ?d.
- (usado ?p - plato): Marca que el plato ?p ya ha sido utilizado en otro día de la semana (para evitar repeticiones).

Tipología:

Estos predicados representan la categoría gastronómica de cada plato (carne, pescado, sopa, etc.) y registran qué tipo se ha usado cada día. Nos sirve para poder tener variedad de tipo de comida entre dos días consecutivos.

- (tipo ?p - plato ?t - tipo): Establece que el plato ?p pertenece al tipo culinario ?t.
- (tipo_primero_asignado ?d - dia ?t - tipo): Marca que el tipo ?t ha sido asignado como primer plato el día ?d.
- (tipo_segundo_asignado ?d - dia ?t - tipo): Marca que el tipo ?t ha sido asignado como segundo plato el día ?d.

Orden de los días:

Estos predicados permiten ordenar los días y distinguir el lunes del resto.

- (anterior ?d - dia ?d_prev - dia): Indica que el día ?d_prev es el día anterior a ?d. Se utiliza para comprobar restricciones entre días consecutivos.
- (es_lunes ?d - dia): Identifica el día lunes, que se trata de manera especial al no tener día anterior.

Platos obligatorios:

- (plato_obligatorio_primero ?d - dia ?p - plato): Indica que el primer plato ?p debe ser asignado obligatoriamente el día ?d.
- (plato_obligatorio_segundo ?d - dia ?p - plato): Indica que el segundo plato ?p debe ser asignado obligatoriamente el día ?d.

Funciones numéricas:

- (calorias ?p - plato): Representa las calorías asociadas al plato ?p.
- (calorias_dia ?d - dia): Suma total de calorías de los platos asignados al día ?d.
- (precio ?p - plato): Precio del plato ?p para calcular el coste del menú.

- (precio_día ?d - día): Suma de los precios de los platos asignados al día ?d.
- (precio_total): Suma del precio de todos los menús de la semana, usada para minimizar el coste semanal.

2.3. Acciones

- **asignar_menu_lunes:** Esta acción se encarga de asignar un primer y un segundo plato para el lunes, el único día que no tiene día anterior. Esto justifica su tratamiento separado del resto de días de la semana.

Permite iniciar la planificación semanal sin depender de restricciones entre días. Como no hay día anterior, se evita evaluar restricciones de tipo repetido en días consecutivos (que solo aplican de martes en adelante).

Precondiciones: Requiere que el día sea lunes, que los platos sean un primero y un segundo compatibles entre sí, que no se hayan usado aún en la semana y que el menú del día no esté ya asignado. También controla que la suma de calorías esté entre 1000 y 1500.

Efectos: Se marca el día como asignado, registra los platos como usados, guarda su tipo para evitar repeticiones en días siguientes, calcula las calorías del día, el precio del menú y suma ese coste al total semanal.

- **asignar_menu:** Esta acción asigna platos al resto de los días (de martes a viernes), teniendo en cuenta el día anterior para aplicar restricciones adicionales de variedad.

Precondiciones: Requiere que los platos sean compatibles, que no se hayan usado previamente, que el menú del día no esté ya asignado y que la suma de calorías esté entre 1000 y 1500. Además, comprueba que el tipo del primer y segundo plato no se repita respecto al día anterior, asegurando variedad entre días consecutivos.

Efectos: Los mismos que los de asignar_menu_lunes.

3. Modelado de los problemas a resolver

El modelado de los problemas concretos en PDDL ha sido un paso fundamental para demostrar la capacidad del dominio que hemos ido desarrollando a lo largo de la práctica. Cada problema representa una instancia específica en la que se ponen a prueba las restricciones y funcionalidades incorporadas en el dominio, permitiéndonos validar que el sistema responde correctamente ante diferentes escenarios.

3.1. Objetos

Para cada problema, se definen explícitamente todos los objetos necesarios:

- **Días:** los días de la semana a planificar, en nuestro caso, de lunes a viernes.
- **Platos:** se listan todos los platos disponibles, tanto primeros como segundos, asegurando una variedad suficiente para cubrir todas las combinaciones posibles.
- **Tipos:** se incluyen los distintos tipos culinarios (sopa, carne, pescado, pasta, etc.) para poder controlar las restricciones relativas a la variedad y la no repetición de tipos.

3.2. Estado inicial

El estado inicial recoge toda la información de partida necesaria para que el planificador pueda generar una solución válida:

- **Clasificación de platos:** se indica mediante los predicados (`es_primer` ...) y (`es_segundo` ...) qué platos pueden ser seleccionados como primeros y cuáles como segundos.
- **Tipos de platos:** se asocia cada plato a su tipo culinario mediante el predicado (`tipo ?p ?t`).
- **Compatibilidades:** se especifican las parejas de platos incompatibles con el predicado (`incompatible ?p1 ?p2`).
- **Valores numéricos:** se asignan calorías y precios a cada plato con las funciones (`= (calorias ?p) ...`) y (`= (precio ?p) ...`), y se inicializa el precio total a cero.
- **Relaciones entre días:** se define el orden de los días con el predicado (`anterior ?d ?d_prev`) y se marca el lunes como día de inicio con (`es_lunes lunes`).

- **Platos obligatorios:** se indica qué platos deben aparecer obligatoriamente en determinados días usando los predicados (`plato_obligatorio_primero ?d ?p`) y (`plato_obligatorio_segundo ?d ?p`).
- **Menús a planificar:** se indica que cada día requiere un menú con (`menu_dia ?d`).

3.3. Estado final

El objetivo del problema es que todos los días tengan un menú asignado cumpliendo todas las restricciones impuestas:

- Se exige que cada día tenga su menú asignado mediante (`menu-asignado ?d`).
- Se comprueba que los platos obligatorios estén presentes en los días correspondientes, tanto para primeros como para segundos, utilizando cuantificadores universales y existenciales.
- El objetivo global queda expresado en una conjunción de condiciones, asegurando que todas las restricciones del dominio se cumplen en la solución final.
- En la extensión 5, se incluye además la métrica de optimización: (`:metric minimize (precio_total)`). De este modo, el planificador no sólo debe encontrar una solución válida, sino que también debe minimizar el coste total del menú semanal generado.

Como es evidente, el modelado del problema se ha ido adaptando de manera progresiva, partiendo de una definición simple en el nivel básico y ampliándose hasta abarcar todas las restricciones y objetivos de la extensión 5. En cada caso, se han modificado los objetos, el estado inicial y el objetivo/estado final, adaptándolos a los requisitos específicos de cada extensión para poner a prueba y demostrar la capacidad del dominio desarrollado.

4. Desarrollo de los modelos

Una vez establecido el contexto del enunciado de la práctica, hemos optado por realizar el desarrollo de los modelos de forma iterativa, es decir, siguiendo las pautas y requisitos establecidos en cada extensión. En cada iteración, se ha analizado la nueva restricción o funcionalidad a incorporar y se han adaptado tanto el dominio como los problemas de ejemplo, asegurándonos así de que el sistema evolucionase de forma ordenada y controlada.

El proceso comenzó con la definición del dominio básico, centrado en la compatibilidad entre platos y la asignación de menús diarios. A partir de esa base, se fueron añadiendo progresivamente nuevas restricciones: primero la no repetición de platos, después el control de tipos culinarios en días consecutivos, la inclusión de platos obligatorios en días concretos, el control del rango de calorías diarias y, finalmente, la minimización del precio semanal. Cada una de estas extensiones ha supuesto la introducción de nuevos predicados, funciones y condiciones en las acciones, los cuales quedan reflejados en los cambios y la ampliación del código en las distintas extensiones, mostrando claramente cómo cada iteración aportaba nuevas capacidades al modelo.

En cada etapa, se ha validado el funcionamiento del modelo de forma constante mediante juegos de pruebas específicos, con el fin de detectar y corregir posibles errores a medida que se iba modificando el código. Este enfoque incremental nos ha permitido analizar el impacto de cada cambio sobre el conjunto del modelo y mantener la capacidad de adaptación del sistema, asegurando que las nuevas incorporaciones no afectaran a lo que ya habíamos implementado correctamente.

5. Problemas de prueba

Para conseguir los juegos de prueba, hemos creado por un lado un juego de pruebas por nuestra cuenta, y por otro lado un juego de pruebas a través de un código que hemos hecho con python.

5.1. Nivel básico

Estos juegos de prueba corresponden al nivel básico, donde se pide simplemente la construcción de un menú semanal que asigna un primero y un segundo a cada día, y estos son compatibles entre si.

5.1.1. Primer juego de pruebas

Este es el juego de pruebas que hemos desarrollado por nuestra cuenta, y se ha usado inicialmente para ver si el código desarrollado funcionaba correctamente.

Nombre Plato	Posición
ensalada	Primero
sopa	Primero
paella	Primero
crema	Primero
pasta	Primero
arroz	Primero
pollo	Segundo
merluza	Segundo
lasagna	Segundo
fabada	Segundo
calamar	Segundo
tortilla	Segundo

Incompatibilidades

paella y fabada
sopa y fabada
pasta y calamar

Salida:

```
time spent:    0.15 seconds instantiating 165 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 885 facts and 165
actions
              0.00 seconds creating final representation with 330 relevant facts
              0.00 seconds building connectivity graph
              0.31 seconds searching, evaluating 166 states, to a max depth of 1
              0.46 seconds total time
```

El planificador Fast-Forward ha generado un plan en 0.46 segundos. Durante este tiempo, instanció 165 acciones simples y ninguna compleja, identificó 885 hechos alcanzables y redujo el problema a 330 hechos relevantes. No hubo necesidad de un análisis de conectividad complejo. La búsqueda evaluó 166 estados y encontró una solución en una profundidad máxima de 1, lo que indica un problema bien modelado y una solución directa.

5.1.2. Segundo juego de pruebas

Este es el juego de pruebas después de ejecutar nuestro generador de juegos de prueba.

Nombre	Posición
prim0	primero
prim1	primero
prim2	primero
prim3	primero
prim4	primero
prim5	primero
prim6	primero
seg0	segundo

seg1	segundo
seg2	segundo
seg3	segundo
seg4	segundo
seg5	segundo
seg6	segundo
seg7	segundo
seg8	segundo
seg9	segundo

Tabla de incompatibilidades:

Primero	Segundo
prim2	seg5
prim6	seg2
prim1	seg0
prim6	seg8

Salida:

```
time spent:    0.00 seconds instantiating 330 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 1775 facts and 330
actions
              0.00 seconds creating final representation with 660 relevant facts
              0.00 seconds building connectivity graph
              2.03 seconds searching, evaluating 331 states, to a max depth of 1
              2.03 seconds total time
```

Como podemos ver, al igual que en el caso anterior, se ha encontrado una solución con el fast forward.

5.2. Extensión 1

En esta extensión, además de crear un menú básico, no hay platos repetidos en toda la semana.

5.2.1. Primer juego de pruebas

Este es el juego de pruebas que hemos desarrollado por nuestra cuenta, y se ha usado inicialmente para ver si el código desarrollado funcionaba correctamente.

Nombre Plato	Posición
ensalada	Primero
sopa	Primero
paella	Primero
crema	Primero
pasta	Primero
arroz	Primero
pollo	Segundo
merluza	Segundo
lasagna	Segundo
fabada	Segundo
calamar	Segundo
tortilla	Segundo

Tabla incompatibilidades:

Incompatibilidades
paella y fabada
sopa y fabada

Salida:

```
ff: found legal plan as follows
```

```
step    0: ASIGNAR_MENU LUNES ENSALADA POLLO
        1: ASIGNAR_MENU MARTES SOPA MERLUZA
```

```
2: ASIGNAR_MENU MIERCOLES PAELLA LASAGNA
3: ASIGNAR_MENU JUEVES CREMA FABADA
4: ASIGNAR_MENU VIERNES PASTA CALAMAR
```

```
time spent:    0.00 seconds instantiating 170 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 204 facts and 170
actions
              0.00 seconds creating final representation with 204 relevant facts
              0.00 seconds building connectivity graph
              0.15 seconds searching, evaluating 6 states, to a max depth of 1
              0.15 seconds total time
```

El planificador Fast-Forward ha resuelto el problema en 0.15 segundos. Se instanciaron 170 acciones simples, sin necesidad de acciones complejas. Se identificaron 204 hechos relevantes y no fue necesario un análisis de conectividad costoso. La búsqueda fue altamente eficiente, evaluando sólo 6 estados con una profundidad máxima de 1.

El plan resultante asigna un menú completo para los cinco días laborables:

- Lunes: Ensalada y Pollo
- Martes: Sopa y Merluza
- Miércoles: Paella y Lasagna
- Jueves: Crema y Fabada
- Viernes: Pasta y Calamar

Este resultado demuestra que el modelo es válido ya que no se usa el mismo plato más de una vez a la semana.

5.1.2. Segundo juego de pruebas

Este es el juego de pruebas después de ejecutar nuestro generador de juegos de prueba.

Tabla con información de los platos:

Nombre	Posición	Tipo
prim0	primero	vegetariano
prim1	primero	huevo
prim2	primero	huevo
prim3	primero	carne

prim4	primero	pescado
prim5	primero	carne
prim6	primero	arroz
prim7	primero	sopa
seg0	segundo	huevo
seg1	segundo	carne
seg2	segundo	huevo
seg3	segundo	huevo
seg4	segundo	sopa
seg5	segundo	pescado

Tabla de incompatibilidades:

Primero	Segundo
prim5	seg4
prim0	seg1
prim0	seg2
prim7	seg3

Este ha sido el resultado:

ff: found legal plan as follows

```

step    0: ASIGNAR_MENU LUNES PRIM0 SEG0
        1: ASIGNAR_MENU MARTES PRIM1 SEG1
        2: ASIGNAR_MENU MIERCOLES PRIM2 SEG2
        3: ASIGNAR_MENU JUEVES PRIM3 SEG3
        4: ASIGNAR_MENU VIERNES PRIM4 SEG4

```

```

time spent:    0.15 seconds instantiating 220 easy, 0 hard action templates
               0.00 seconds reachability analysis, yielding 258 facts and 220
actions
               0.00 seconds creating final representation with 258 relevant facts
               0.00 seconds building connectivity graph

```

```
0.00 seconds searching, evaluating 6 states, to a max depth of 1
0.15 seconds total time
```

Podemos ver que, efectivamente, encuentra una solución óptima y no se repite ningún plato durante la semana. Tarda solo 0.15 segundos, y se evalúan 6 estados para encontrar la solución.

5.3. Extensión 2

En esta extensión, además de las restricciones previas, hay que mirar que ningún tipo de plato se repita (se mira por separado primero y segundo).

5.3.1. Primer juego de pruebas

Este es el juego de pruebas que hemos desarrollado por nuestra cuenta, y se ha usado inicialmente para ver si el código desarrollado funcionaba correctamente.

Nombre Plato	Posición
ensalada	Primero
sopa	Primero
paella	Primero
crema	Primero
pasta	Primero
arroz	Primero
pollo	Segundo
merluza	Segundo
lasagna	Segundo
fabada	Segundo
calamar	Segundo
tortilla	Segundo

Incompatibilidades
paella y fabad
sopa y fabada

pasta y calamar

Salida:

ff: found legal plan as follows

```
step    0: ASIGNAR_MENU LUNES ENSALADA POLLO
        1: ASIGNAR_MENU MARTES SOPA MERLUZA
        2: ASIGNAR_MENU MIERCOLES PAELLA LASAGNA
        3: ASIGNAR_MENU JUEVES CREMA FABADA
        4: ASIGNAR_MENU VIERNES PASTA CALAMAR
```

```
time spent:  0.00 seconds instantiating 170 easy, 0 hard action templates
            0.00 seconds reachability analysis, yielding 204 facts and 170
actions
            0.00 seconds creating final representation with 204 relevant facts
            0.00 seconds building connectivity graph
            0.15 seconds searching, evaluating 6 states, to a max depth of 1
            0.15 seconds total time
```

El planificador Fast-Forward ha resuelto el problema en 0.15 segundos. Se instanciaron 170 acciones simples, sin necesidad de acciones complejas. Se identificaron 204 hechos relevantes y no fue necesario un análisis de conectividad costoso. La búsqueda fue altamente eficiente, evaluando solo 6 estados con una profundidad máxima de 1.

El plan resultante asigna un menú completo para los cinco días laborables:

- Lunes: Ensalada y Pollo
- Martes: Sopa y Merluza
- Miércoles: Paella y Lasagna
- Jueves: Crema y Fabada
- Viernes: Pasta y Calamar

Este resultado demuestra que el modelo es válido, cumple las restricciones establecidas y produce soluciones rápidas y coherentes.

5.3.2. Segundo juego de pruebas

Este es el juego de pruebas después de ejecutar nuestro generador de juegos de prueba.

Tabla con la información de los platos:

Nombre	Posición	Tipo
--------	----------	------

prim0	primero	vegetariano
prim1	primero	huevo
prim2	primero	huevo
prim3	primero	carne
prim4	primero	pescado
prim5	primero	carne
prim6	primero	arroz
prim7	primero	sopa
seg0	segundo	huevo
seg1	segundo	carne
seg2	segundo	huevo
seg3	segundo	huevo
seg4	segundo	sopa
seg5	segundo	pescado

Tabla de incompatibilidades:

Primero	Segundo
prim5	seg4
prim0	seg1
prim0	seg2
prim7	seg3

Este ha sido el resultado de la ejecución:

ff: found legal plan as follows

```
step    0: ASIGNAR_MENU_LUNES LUNES PRIM0 SEG0 VEGETARIANO HUEVO
        1: ASIGNAR_MENU MARTES PRIM1 SEG1 HUEVO CARNE LUNES
        2: ASIGNAR_MENU MIERCOLES PRIM3 SEG2 CARNE HUEVO MARTES
        3: ASIGNAR_MENU JUEVES PRIM2 SEG4 HUEVO SOPA MIERCOLES
        4: ASIGNAR_MENU VIERNES PRIM4 SEG3 PESCADO HUEVO JUEVES
```

```
time spent:    0.00 seconds instantiating 220 easy, 0 hard action templates
               0.00 seconds reachability analysis, yielding 368 facts and 220
actions
               0.00 seconds creating final representation with 358 relevant facts
               0.00 seconds building connectivity graph
               0.16 seconds searching, evaluating 6 states, to a max depth of 1
               0.16 seconds total time
```

El planificador FF resolvió correctamente el problema, asignando un menú válido para cada día laboral. Se respetaron las restricciones de tipos, la relación entre días y las incompatibilidades entre platos. Vemos que en 2 días consecutivos no se repite el mismo tipo de plato (se mira diferente en primeros y segundos).

5.4. Extensión 3

Además de las restricciones anteriores, ahora pueden haber platos que se sirvan un día en concreto.

5.4.1. Primer juego de pruebas

Este ha sido el juego de pruebas que hemos hecho nosotros.

Tabla de información de cada plato:

Nombre	Posición	Tipo
ensalada	primero	ensalada
sopa	primero	sopa
paella	primero	arroz
crema	primero	sopa
pasta	primero	pasta

arroz	primero	arroz
pollo	segundo	carne
merluza	segundo	pescado
lasagna	segundo	pasta
fabada	segundo	carne
calamar	segundo	marisco
tortilla	segundo	huevo

Tablas de incompatibilidades (izq.) y tabla de platos obligatorios (der.) :

Primero	Segundo
paella	fabada
sopa	fabada

Día	Tipo de plato	Plato
lunes	Primero	crema
jueves	Primero	paella
viernes	Segundo	lasagna

Este ha sido el resultado obtenido:

ff: found legal plan as follows

```

step    0: ASIGNAR_MENU MIERCOLES SOPA MERLUZA SOPA PESCADO MARTES
        1: ASIGNAR_MENU MARTES PASTA FABADA PASTA CARNE LUNES
        2: ASIGNAR_MENU_LUNES LUNES CREMA CALAMAR SOPA MARISCO
        3: ASIGNAR_MENU_VIERNES ENSALADA LASAGNA ENSALADA PASTA JUEVES
        4: ASIGNAR_MENU JUEVES PAELLA POLLO ARROZ CARNE MIERCOLES
        5: REACH-GOAL

```

```

time spent:    0.15 seconds instantiating 170 easy, 0 hard action templates
               0.00 seconds reachability analysis, yielding 329 facts and 350
actions
               0.00 seconds creating final representation with 295 relevant facts
               0.16 seconds building connectivity graph
               0.00 seconds searching, evaluating 28 states, to a max depth of 1
               0.31 seconds total time

```

El planificador FF resolvió con éxito el problema modelado para la Extensión 3, generando un plan legal que respeta todos los requisitos del dominio. El tiempo total de ejecución fue de sólo 0.31 segundos, lo que demuestra la eficiencia del modelo. Además, se observa que se asignaron los platos requeridos para cada día, cosa que demuestra la corrección de nuestra solución.

5.4.2. Segundo juego de pruebas

Nombre	Posición	Tipo
prim0	primero	sopa
prim1	primero	huevo
prim2	primero	huevo
prim3	primero	arroz
prim4	primero	sopa
prim5	primero	arroz
prim6	primero	arroz
prim7	primero	carne
seg0	segundo	vegetariano
seg1	segundo	arroz
seg2	segundo	pescado
seg3	segundo	carne
seg4	segundo	arroz
seg5	segundo	arroz
seg6	segundo	arroz
seg7	segundo	carne
seg8	segundo	vegetariano
seg9	segundo	huevo

Tablas de incompatibilidades (izq.) y tabla de platos obligatorios (der.) :

Primero	Segundo
prim5	seg2
prim7	seg8

Día	Tipo de plato	Plato
jueves	Primero	prim5
jueves	Segundo	seg0

prim7	seg5
prim2	seg8

Este ha sido el resultado:

ff: found legal plan as follows

```

step    0: ASIGNAR_MENU VIERNES PRIM0 SEG1 SOPA ARROZ JUEVES
        1: ASIGNAR_MENU MIERCOLES PRIM1 SEG2 HUEVO PESCADO MARTES
        2: ASIGNAR_MENU MARTES PRIM2 SEG3 HUEVO CARNE LUNES
        3: ASIGNAR_MENU_LUNES LUNES PRIM3 SEG4 ARROZ ARROZ
        4: ASIGNAR_MENU JUEVES PRIM5 SEG0 ARROZ VEGETARIANO MIERCOLES
        5: REACH-GOAL

```

```

time spent:    0.00 seconds instantiating 380 easy, 0 hard action templates
               0.00 seconds reachability analysis, yielding 531 facts and 452
actions
               0.00 seconds creating final representation with 517 relevant facts
               0.00 seconds building connectivity graph
               0.00 seconds searching, evaluating 92 states, to a max depth of 1
               0.00 seconds total time

```

Podemos ver que el código ha encontrado una buena solución cumpliendo todas las restricciones propias de la Extensión 3: menús completos, platos obligatorios, tipos no repetidos entre días consecutivos e incompatibilidades. En especial, vemos que el jueves cuenta con sus 2 platos obligatorios.

5.5. Extensión 4

Además de las restricciones anteriores, ahora cada menú tiene que estar nutricionalmente balanceado, por lo que hay que añadir calorías a cada plato.

5.5.1. Primer juego de pruebas

Este ha sido el juego de pruebas que hemos hecho nosotros.

Tabla de información de cada plato:

Nombre	Posición	Tipo	Calorías
ensalada	primero	ensalada	300
sopa	primero	sopa	250
paella	primero	arroz	700
crema	primero	sopa	450

pasta	primero	pasta	500
arroz	primero	arroz	650
pollo	segundo	carne	600
merluza	segundo	pescado	400
lasagna	segundo	pasta	800
fabada	segundo	carne	900
calamar	segundo	marisco	550
tortilla	segundo	huevo	400

Tablas de incompatibilidades (izq.) y tabla de platos obligatorios (der.) :

Primero	Segundo
paella	fabada
sopa	fabada

Día	Tipo de plato	Plato
lunes	Primero	crema
jueves	Primero	paella
viernes	Segundo	lasagna

El resultado obtenido ha sido el siguiente:

ff: found legal plan as follows

```

step    0: ASIGNAR_MENU VIERNES ENSALADA LASAGNA ENSALADA PASTA JUEVES
        1: ASIGNAR_MENU JUEVES PAELLA TORTILLA ARROZ HUEVO MIERCOLES
        2: ASIGNAR_MENU_LUNES LUNES CREMA POLLO SOPA CARNE
        3: ASIGNAR_MENU MIERCOLES PASTA CALAMAR PASTA MARISCO MARTES
        4: ASIGNAR_MENU MARTES ARROZ MERLUZA ARROZ PESCADO LUNES
        5: REACH-GOAL

```

```

time spent:    0.15 seconds instantiating 170 easy, 0 hard action templates
               0.00 seconds reachability analysis, yielding 329 facts and 285
actions
               0.15 seconds creating final representation with 295 relevant facts,
5 relevant fluents
               0.00 seconds computing LNF
               0.00 seconds building connectivity graph
               0.00 seconds searching, evaluating 198 states, to a max depth of 0
               0.30 seconds total time

```

El planificador resolvió correctamente el problema de la Extensión 4, generando un plan que asigna menús válidos y equilibrados en calorías para toda la semana. El resultado

demuestra que el dominio y problema están correctamente modelados para cumplir con las restricciones calóricas, de tipos y de platos obligatorios. El plan fue hallado en apenas 0.30 segundos, con excelente eficiencia.

5.5.2. Segundo juego de pruebas

Este ha sido el juego de pruebas que hemos hecho nosotros.

Tabla de información de cada plato:

Nombre	Posición	Tipo	Calorías
prim0	primero	vegetariano	526
prim1	primero	pescado	521
prim2	primero	carne	408
prim3	primero	huevo	302
prim4	primero	pescado	457
prim5	primero	vegetariano	544
prim6	primero	vegetariano	295
prim7	primero	sopa	551
prim8	primero	vegetariano	431
prim9	primero	huevo	533
seg0	segundo	huevo	651
seg1	segundo	huevo	709
seg2	segundo	pescado	455
seg3	segundo	sopa	723
seg4	segundo	pescado	459
seg5	segundo	carne	468
seg6	segundo	carne	525
seg7	segundo	sopa	783
seg8	segundo	huevo	644

Tablas de incompatibilidades (izq.) y tabla de platos obligatorios (der.) :

Primero	Segundo
prim9	seg4
prim8	seg0
prim7	seg7
prim6	seg6
prim5	seg0

Día	Tipo de plato	Plato
martes	Primero	prim8
martes	Segundo	seg7

El resultado obtenido ha sido el siguiente:

ff: found legal plan as follows

```

step      0: ASIGNAR_MENU VIERNES PRIM9 SEG8 HUEVO HUEVO JUEVES
          1: ASIGNAR_MENU JUEVES PRIM0 SEG3 VEGETARIANO SOPA MIERCOLES
          2: ASIGNAR_MENU MIERCOLES PRIM1 SEG0 PESCADO HUEVO MARTES
          3: ASIGNAR_MENU MARTES PRIM8 SEG7 VEGETARIANO SOPA LUNES
          4: ASIGNAR_MENU_LUNES LUNES PRIM2 SEG1 CARNE HUEVO
          5: REACH-GOAL

time spent: 0.00 seconds instantiating 425 easy, 0 hard action templates
           0.00 seconds reachability analysis, yielding 578 facts and 342
actions
           0.00 seconds creating final representation with 564 relevant facts,
5 relevant fluents
           0.00 seconds computing LNF
           0.16 seconds building connectivity graph
           0.00 seconds searching, evaluating 566 states, to a max depth of 0
           0.16 seconds total time

```

El planificador Metric-FF resolvió correctamente el problema, generando un plan válido que cumple con todas las restricciones definidas en la Extensión 5. El plan fue encontrado de manera rápida (0.16 segundos), evaluando 566 estados y mostrando un modelo eficiente. Se garantiza que los platos asignados son compatibles, variados, y cumplen con los requisitos calóricos.

5.6. Extensión 5

Además de las restricciones anteriores, ahora contamos con un precio en cada plato, y hay que minimizar ese precio.

5.6.1. Primer juego de pruebas

Este ha sido el juego de pruebas que hemos hecho nosotros.

Tabla de información de cada plato:

Nombre	Posición	Tipo	Calorías	Precio
ensalada	primero	ensalada	300	3
sopa	primero	sopa	250	2
paella	primero	arroz	700	5
crema	primero	sopa	450	3
pasta	primero	pasta	500	4
arroz	primero	arroz	650	3
gazpacho	primero	sopa	200	3
sopa_bolonesa	primero	sopa	350	4
bruschetta	primero	vegetariano	300	4
gazpacho_vegano	primero	vegetariano	180	3
pollo	segundo	carne	600	4
merluza	segundo	pescado	400	5
lasagna	segundo	pasta	800	6
fabada	segundo	carne	900	1
calamar	segundo	marisco	550	6
tortilla	segundo	huevo	400	3
tortilla_caducada	segundo	huevo	400	10
quiche	segundo	huevo	600	5
salmon	segundo	pescado	450	6
tortilla_espanola	segundo	huevo	400	4
risotto	segundo	arroz	550	5
hamburguesa	segundo	carne	700	7
ceviche	segundo	marisco	350	3
curry	segundo	carne	650	6

Tablas de incompatibilidades (izq.) y tabla de platos obligatorios (der.) :

Primero	Segundo
paella	fabada
sopa	fabada

Día	Tipo de plato	Plato
lunes	Primero	crema
jueves	Primero	paella
viernes	Segundo	lasagna

El resultado obtenido ha sido el siguiente:

ff: found legal plan as follows

```

step    0: ASIGNAR_MENU VIERNES SOPA LASAGNA SOPA PASTA JUEVES
        1: ASIGNAR_MENU JUEVES PAELLA TORTILLA ARROZ HUEVO MIERCOLES
        2: ASIGNAR_MENU_LUNES LUNES CREMA QUICHE SOPA HUEVO
        3: ASIGNAR_MENU MIERCOLES ARROZ CEVICHE ARROZ MARISCO MARTES
        4: ASIGNAR_MENU MARTES ENSALADA FABADA ENSALADA CARNE LUNES
        5: REACH-GOAL

```

```

time spent:  14.68 seconds instantiating 690 easy, 0 hard action templates
            0.00 seconds reachability analysis, yielding 893 facts and 2100
actions
            7.66 seconds creating final representation with 859 relevant facts,
11 relevant fluents
            0.00 seconds computing LNF
            0.00 seconds building connectivity graph
            0.47 seconds searching, evaluating 627 states, to a max depth of 0
            22.81 seconds total time

```

El planificador Metric-FF resolvió correctamente el problema optimizando el coste total del menú semanal. Reconoció la métrica de minimización y generó un plan legal que asigna menús a todos los días respetando todas las restricciones. Aunque la fase de instanciación fue costosa por la cantidad de acciones, la planificación fue eficiente, encontrando una solución en 22.81 segundos en total. Además, se puede observar que el resultado es correcto ya que los precios obtenidos siempre son bastante bajos respecto los alimentos que no se seleccionan.

5.6.2. Segundo juego de pruebas

Este ha sido el juego de pruebas generado por nuestro código.

Tabla de información de cada plato:

Nombre	Posición	Tipo	Calorías	Precio
--------	----------	------	----------	--------

prim0	primero	arroz	393	5
prim1	primero	huevo	364	5
prim2	primero	huevo	465	2
prim3	primero	arroz	251	2
prim4	primero	arroz	422	3
prim5	primero	arroz	484	2
prim6	primero	carne	561	4
prim7	primero	pescado	408	4
prim8	primero	carne	315	4
prim9	primero	sopa	548	5
prim10	primero	huevo	468	4
prim11	primero	pescado	414	4
prim12	primero	huevo	476	3
prim13	primero	sopa	263	5
prim14	primero	sopa	355	3
seg0	segundo	pescado	491	3
seg1	segundo	pescado	392	6
seg2	segundo	vegetariano	394	6
seg3	segundo	vegetariano	665	4
seg4	segundo	pescado	524	6
seg5	segundo	huevo	514	5
seg6	segundo	arroz	707	3
seg7	segundo	huevo	621	5
seg8	segundo	pescado	741	3
seg9	segundo	vegetariano	535	6
seg10	segundo	arroz	687	6
seg11	segundo	huevo	753	4
seg12	segundo	arroz	617	4

Tablas de incompatibilidades (izq.) y tabla de platos obligatorios (der.) :

Primero	Segundo
prim3	seg3
prim3	seg10
prim14	seg11
prim8	seg7
prim5	seg0

Día	Tipo de plato	Plato
miercoles	Primero	prim6
miercoles	Segundo	seg11

Este ha sido el resultado:

ff: found legal plan as follows

```

step    0: ASIGNAR_MENU VIERNES PRIM2 SEG10 HUEVO ARROZ JUEVES
        1: ASIGNAR_MENU MIERCOLES PRIM6 SEG11 CARNE HUEVO MARTES
        2: ASIGNAR_MENU JUEVES PRIM5 SEG6 ARROZ ARROZ MIERCOLES
        3: ASIGNAR_MENU MARTES PRIM4 SEG12 ARROZ ARROZ LUNES
        4: ASIGNAR_MENU_LUNES LUNES PRIM12 SEG8 HUEVO PESCADO
        5: REACH-GOAL

```

```

time spent:    0.00 seconds instantiating 950 easy, 0 hard action templates
               0.00 seconds reachability analysis, yielding 1121 facts and 682
actions
               0.15 seconds creating final representation with 1107 relevant facts,
11 relevant fluents
               0.00 seconds computing LNF
               0.00 seconds building connectivity graph
               0.15 seconds searching, evaluating 1098 states, to a max depth of 0
               0.30 seconds total time

```

Lo que indica este resultado es que El planificador Metric-FF resolvió correctamente el problema asociado a la Extensión 5, minimizando el valor del fluente (precio_total). La función objetivo fue reconocida y normalizada como minimize (precio_total), y se generó un plan válido que asigna menús a cada día de la semana respetando todas las restricciones. Se puede observar que el resultado es bastante acertado ya que los precios obtenidos son siempre bastante bajos (excepto en 1 caso, siempre están por debajo de 4).

El planificador instanció 950 acciones, evaluó 1098 estados y encontró la solución en 0.30 segundos, demostrando tanto la eficiencia del modelo como la correcta implementación del criterio de optimización.